



OPEN DSS-YOLO: an improved lightweight real-time fire detection model based on YOLOv8

Hongjie Wang, Xiaoyang Fu✉, Zixuan Yu & Zhifeng Zeng

Fire disasters pose significant risks to human life, economic development, and social stability. The early stages of a fire, often characterized by small flames, diffuse smoke, and obstructed objects, can lead to challenges such as missed detections and poor real-time performance. To address these issues, we propose a DSS-YOLO model based on an improved YOLOv8n architecture, designed to enhance the recognition accuracy of obscured objects and small targets while reducing computational overhead. Specifically, we replace all C2f modules in the Backbone with DynamicConv modules to reduce computation without sacrificing feature extraction capabilities. We also introduce the SEAM attention mechanism to improve detection of obscured and small targets, and the SPPELAN module at the end of the Backbone to enhance detection across different scales. The model is evaluated using the public dataset mytest-hrswj, which contains diverse fire scenarios, including indoors, forests, and buildings. Compared with the original YOLOv8n, the DSS-YOLOv8 model proposed in this paper improves mAP by 0.6% and Recall by 1.6%, while reducing the model size and FLOPs by 3.4% and 12.3% respectively. The results of this study provide effective technical support for intelligent fire monitoring systems, significantly reducing the computational cost of the model. It enhances real-time fire detection capabilities in complex fire scenarios, facilitating the early detection of fire hazards and helping to minimize the damage caused by fires.

Keywords Fire detection, YOLOv8, DynamicConv, SEAM, SPPELAN

Fire disasters in residential buildings are often the result of human activities and are classified as events that cause significant property loss and endanger human health and life. The fire response performance in the event of a building fire is determined by the characteristics of fire, human factors, and building features. Globally, fires cause over 300,000 deaths each year, and millions suffer permanent injuries¹. In China, from January to October 2023, there were approximately 745,000 reported fires nationwide, resulting in 1,381 deaths and 2,063 injuries, with direct property losses amounting to 6.15 billion yuan. Compared to the same period last year, the number of incidents and injuries increased by 2.5% and 6.5%, respectively².

Existing detection methods rely on various sensors^{3,4}, such as smoke sensors⁵, heat sensors⁶, gas sensors⁷, and flame sensors⁸. These devices are typically low-cost, simple in technology, miniaturized, and easy to install, but they have a short effective sensing range and require manual confirmation of actual fire events. For example, in complex environments such as workshop production or mining operations, temperature sensors can be interfered with by localized heat generated during production, while smoke sensors may be affected by dust produced in the process. In contrast, vision-based detection can effectively avoid the interference caused by localized heating and dust by capturing and analyzing environmental images⁹.

Fire is indeed one of the main causes of casualties, property damage, and economic disruption. Each year, numerous fire incidents occur worldwide, resulting in devastation that is difficult to quantify and describe. To minimize the impact of fires, implementing innovative and effective early warning technologies is crucial¹⁰. With the development of computer vision and deep learning technologies¹¹, image and video-based fire detection methods have increasingly become mainstream. These methods leverage computer vision techniques for real-time monitoring and analysis, effectively identifying fires in a wider range of scenarios and reducing false alarm rates. Additionally, convolutional neural networks (CNN) and object detection algorithms, such as YOLO, can be used for the recognition of flames and smoke. For example:

Zhang et al.¹² improved detection accuracy by replacing the original C2f module with FasterNet, incorporating the CBAM attention mechanism, and adding a detection head for small objects of size 160x160. They also introduced EIOU as the loss function for bounding box regression. These modifications enhanced

School of Computer Science, Zhuhai College of Science and Technology, Ankee Road East, Zhuhai 519040, Guangdong, China. ✉email: fxy@zcsst.edu.cn

detection accuracy and made the model more lightweight, though they also increased its complexity. Wang¹³ addressed the problem of information loss in fire detection within the YOLOv9 architecture by introducing the PGI algorithm, adopting a new lightweight network structure called GELAN, and integrating Focaler-IoU, thereby enhancing the model's accuracy for complex environments but also adding to its complexity. Zhao¹⁴ expanded the feature extraction network, enhancing feature propagation in three dimensions and optimizing the feature pyramid to obtain the best prediction boxes, which resulted in excellent performance in fire detection under various natural lighting conditions; however, the accuracy remains low for partially occluded objects. Xue¹⁵ improved the performance of forest fire small target detection by modifying the Backbone layer of YOLOv5, introducing the CBAM attention module, optimizing the Neck layer structure, and adding a small target detection layer. However, the increase in model parameters resulted in reduced real-time performance. Shao¹⁶ significantly enhanced YOLOv7's performance in small fire and smoke detection tasks by incorporating the SPD-Conv module, replacing PANet with the BiFPN structure, optimizing ELAN-W into the C3 module, and improving the loss function to Focal-CIoU. Nonetheless, the model's computational cost remains high, and it is sensitive to environmental factors (e.g., occlusion), which may lead to false alarms.

Recently, this paper examined the current state of fire detection, tested mainstream deep learning models and open-source datasets, and made algorithmic improvements to certain models. This paper specifically addresses the challenges of detecting small, obscured, and time-sensitive fire targets by modifying the YOLOv8n model¹⁷ to enhance its detection capability for fire targets. Furthermore, the YOLOv8n model has a small number of parameters and is computationally lightweight, making it suitable for future deployment on mobile and embedded devices, which is why this model was chosen for the study.

In fire scenes, fire targets are usually small, complex in shape, and may be occluded by buildings, trees, or other objects, leading to partial loss of target features. Traditional detection models often fail to accurately identify these smaller and occluded targets. Therefore, this paper addresses these challenges with the following research efforts to improve efficient detection in fire scenes.

The remainder of this paper is organized as follows: section “Materials and methods” briefly introduces YOLOv8, describes the modified model algorithms, and explains each module. Section “Results” analyzes and compares the improved DSS-YOLO model through experiments. Section “Discussion” summarizes the entire research work.

Materials and methods

YOLOv8

YOLOv8 is a next-generation object detection algorithm developed by Ultralytics based on YOLOv5. It has achieved significant improvements in both accuracy and speed. YOLOv8 not only supports object detection but also extends to tasks such as image classification and instance segmentation, making it a more versatile framework for computer vision. YOLOv8 mainly consists of three components: the Backbone, which is responsible for extracting deep features from the image as input to subsequent detection modules; the Neck, which is responsible for the fusion and enhancement of features extracted by the Backbone; and the Head, which performs localization and classification of the objects based on the feature maps produced by the Neck.

Compared with other object detection models, such as Faster-RCNN, YOLOv8 offers superior speed and excels in real-time detection and resource-constrained environments, making it particularly suitable for applications requiring rapid response¹⁸. Compared to previous YOLO versions, YOLOv8 significantly improves both speed and accuracy through architectural enhancements, loss function optimization, and multi-task support, demonstrating exceptional performance in small object detection and multi-task scenarios. Additionally, its user-friendly design and flexible deployment make YOLOv8 more efficient and convenient for practical applications^{19,20}.

DynamicConv

Since convolution operations typically generate a large number of parameters, these operations often lead to large model sizes and high computational complexity. To address this issue, this paper uses DynamicConv²¹, which is implemented using ParameterNet. The core concept of ParameterNet involves using dynamic convolution as a key component, mapping input features to multiple expert convolution kernels and dynamically adjusting the weight of each kernel based on the input. This allows ParameterNet to significantly increase the number of parameters in the network without increasing computational cost, thereby improving network performance.

Compared to the traditional convolutional layer, ParameterNet utilizes a dynamic convolution layer named DynamicConv, as illustrated in Fig. 1. DynamicConv consists of two main components: the routing layer and the conditional convolution layer. The input feature maps of the routing layer are pooled and flattened into a vector, which is then fed into a fully connected layer to generate weights for multiple experts. These weights are used to dynamically select and combine multiple convolutional kernels, with the Sigmoid activation function being applied.

DynamicConv is the core component of ParameterNet. It increases the number of model parameters by introducing multiple dynamic experts. The mathematical formulation of dynamic convolution is as follows:

$$Y = X * W' \quad (1)$$

$$W' = \sum_{i=1}^M \alpha_i W_i \quad (2)$$

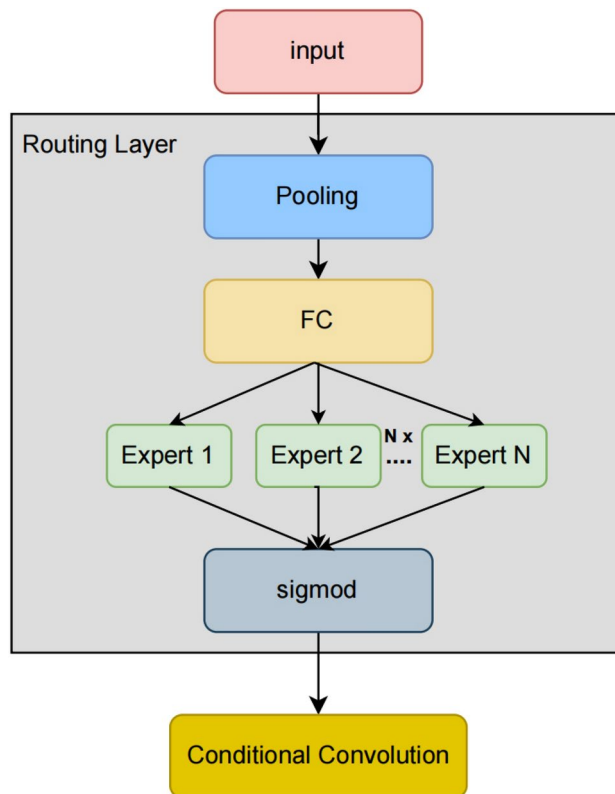


Fig. 1. DynamicConv structure.

Where W_i is the i -th convolution weight tensor, and α_i is the corresponding dynamic coefficient. The dynamic coefficient α is dynamically generated based on the input sample X , and a multi-layer perceptron (MLP) module is usually used to generate these coefficients. The mathematical expression for generating the coefficient is:

$$\alpha = \text{softmax}(\text{MLP}(\text{Pool}(X))) \quad (3)$$

$\text{Pool}(X)$ performs a global pooling operation on the input X , and then generates coefficients through two layers of MLP modules and softmax activation function.

SEAM

In order to improve the accuracy of the model in recognizing occluded objects, this paper incorporates the SEAM²² module into the Head part to process the upsampled feature maps and detection branches of different scales. The SEAM network structure is shown in Fig. 2, which includes three CSMM modules of different sizes (patch-6, patch-7, patch-8). The outputs of these modules are average pooled, then expanded (Channel exp) and finally multiplied to provide enhanced feature representation. The structure of CSMM (Channel and Spatial Mixing Module). CSMM uses different patches to process multi-scale features and uses deep separable convolution to learn the correlation between spatial dimensions and channels.

This module design aims to enhance the network's ability to attend to and capture occluded features by carefully handling both the spatial dimensions and the channels. By comprehensively utilizing multi-scale features and depth-separable convolutions, CSMM improves the accuracy of feature extraction while maintaining computational efficiency.

SPPELAN

In order to enhance the accuracy of identifying fire areas in complex backgrounds and to detect fires of varying sizes and developmental stages, this paper introduces SPPELAN, as described in YOLOv9²³. SPPELAN is a combination of Spatial Pyramid Pooling (SPP) and Efficient Layer Aggregation Network (ELAN).

SPPELAN employs a hierarchical aggregation approach, converting feature maps into spatial pyramid representations at various scales through pooling operations of differing sizes. These representations are then stacked and fused to ultimately derive global context information.

SPPF is an early fire target detection module, and SPPELAN is its improved version. Compared to SPPELAN, SPPF primarily focuses on extracting and fusing features at different scales but lacks refined aggregation of local information. As a result, it may perform inadequately when dealing with small targets or occluded objects.

Fire targets are often very small in images and may be occluded by objects. The SPPELAN module enhances the model's adaptability to targets of different scales through pyramid pooling and local information aggregation.

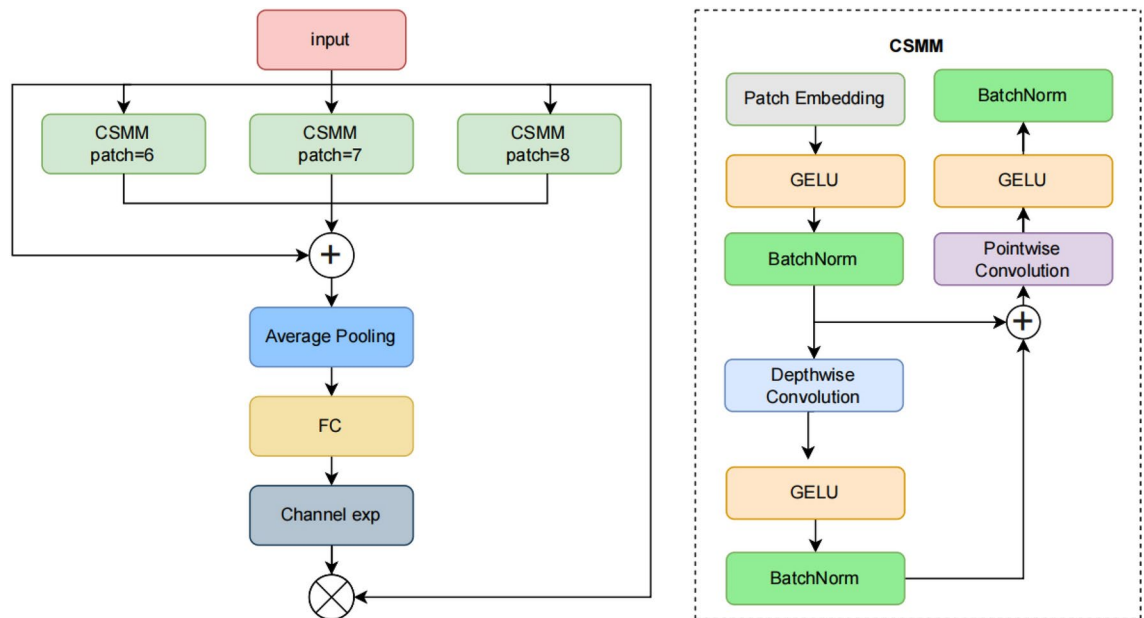


Fig. 2. SEAM network structure diagram.

Pyramid pooling effectively handles features of multi-scale targets, particularly aiding in the detection of small targets. Meanwhile, local information aggregation improves detection capability for small targets, occluded targets, and multiple targets in complex scenes through refined feature learning.

DSS-YOLO

In the early stages of a fire, there are often small flames, accompanied by diffuse smoke and partially blurred objects. Therefore, it is necessary to strengthen the detection of small and occluded targets. At the same time, in order to facilitate subsequent deployment on mobile devices or embedded devices, a lightweight network structure is also required to achieve real-time and accurate detection of fire targets. In order to reduce the amount of model calculation and maintain good feature extraction capabilities, this paper introduces the DynamicConv module to replace all the original C2f modules in Backbone. To improve the detection of occluded objects and prevent a decrease in accuracy, this paper integrates the SEAM attention mechanism at the head, enhancing detection capabilities for small and occluded objects and increasing the model's robustness. Additionally, to boost multi-scale feature extraction and enhance positional accuracy, this paper adopts the SPPELAN template over the traditional SPPF template. The synthesis of the DSS-YOLO model through the integration of DynamicConv, SEAM, and SPPELAN modules is depicted in Fig. 3.

The Backbone section is used to extract the features of the image. It consists of multiple Conv layers and DynamicConv layers. Dynamic convolution can adaptively adjust the convolution kernel according to the features of the input data. The Head part is used to detect targets based on the extracted features. It consists of multiple Conv layers, Upsample layers, Concat layers, SEAM modules and Detect layers. The Upsample and Concat layers merge feature maps from various levels, enhancing the richness of contextual information. The SEAM module is an attention mechanism that improves the model's ability to locate and classify targets. The Detect layer ultimately outputs the category and location information of the target.

This paper replaces the original C2f module in the Backbone section with the DynamicConv module. The primary goal is to significantly increase the number of parameters of the network without increasing the computational cost, thereby enhancing the network's performance. Compared to C2f, DynamicConv does not increase the overall computational cost of the model; hence, the total computational load is actually reduced.

This paper adds the SEAM module to the rear of each Cf2 layer in the HEAD section. Specifically, the SEAM module is positioned at the feature fusion sites after each upsampling, as well as before the prediction layer of each detection head. The aim is to improve the model's ability to recognize occluded objects and enhance its robustness.

The SPPELAN module is added to the end of the Backbone section with the aim of introducing multi-scale information at the top of the feature pyramid, enhancing the model's capability to detect targets of various scales.

Most lightweight models, in order to improve inference speed, typically employ simpler network architectures that overlook the capture of fine-grained features. This results in disadvantages when dealing with small or occluded objects. In contrast, DSS-YOLO incorporates three modules-DynamicConv, SEAM, and SPPELAN-which not only enhance the detection capability for small and occluded objects but also maintain high performance while achieving model lightweight.

The pseudocode for the DSS-YOLO algorithm presented in this paper is outlined in 11 steps: data preparation, dataset division, data augmentation, model initialization, setting hyperparameters, defining the model architecture, training the model, saving the trained model, visualizing images during training, testing the

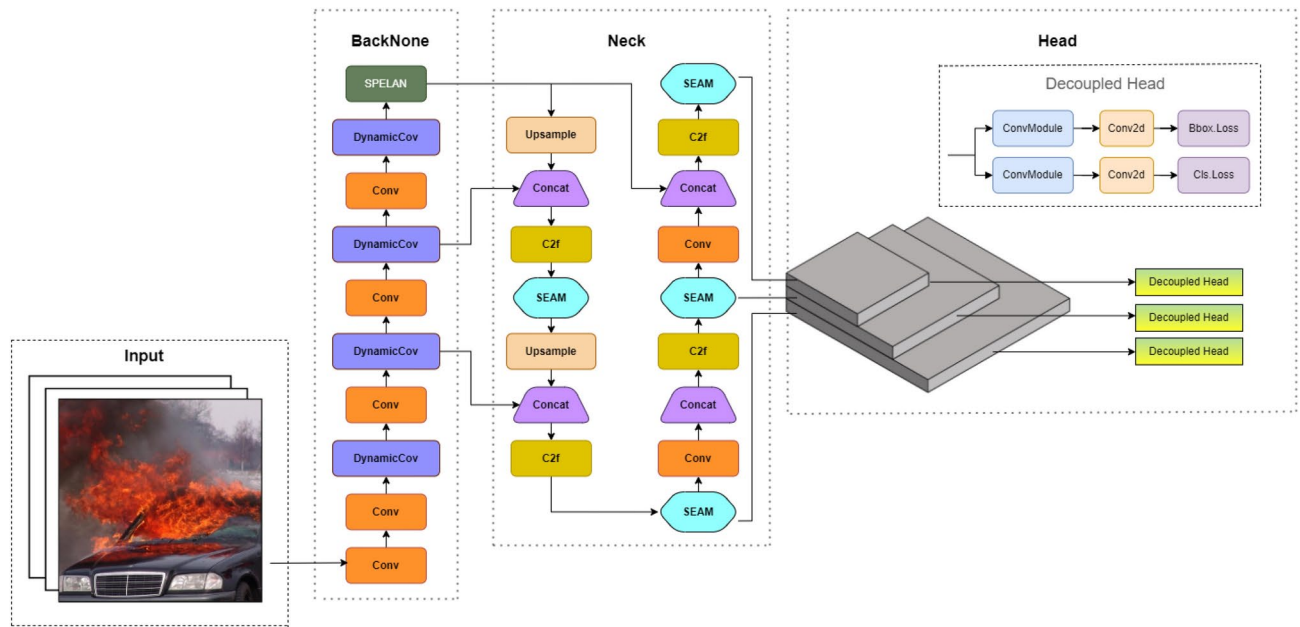


Fig. 3. DSS-YOLO network structure.

model and visualizing the results, and validating the model and visualizing the results. The pseudocode is shown in Appendix A.

Results

Data set

The experimental data set used in this experiment is mytest-hrswj²⁴ provided by the user Mytyolov8 in Roboflow. This data set contains 2384 images and 2384 corresponding annotation files, with a total of one label 'fire'. This data set includes fire situations in various scenes such as indoors, forests, buildings, and lighters, and meets the requirements of being close to real scenes.

This paper randomly divides the dataset into training, validation, and test sets with a ratio of 80%, 10%, and 10%, respectively. To improve the generalization ability of the model, rotation, translation, scaling, shearing, HSV transformation, and Mosaic blending were performed in the preprocessing stage.

Operations such as rotation, translation, and scaling help the model recognize variations of fire targets at different positions, angles, and scales. Shearing and HSV transformations help the model adapt to changes in the shape of fire targets and environmental conditions. Mosaic Augmentation technology, by enhancing training for multi-target scenes, further improves the model's multi-target detection capability in complex environments. All these augmentation techniques lay a solid foundation for DSS-YOLO's efficient performance in fire target detection.

Experimental environment and configuration

The experiment in this paper is built using the PyTorch deep learning framework. The development environment includes Python 3.8.10, PyTorch 2.0.0, and CUDA 11.8, running on the Ubuntu 20.04 operating system. The hardware environment consists of an AMD EPYC 9754 128-Core Processor CPU and an NVIDIA GeForce RTX 3090 (24GB) GPU. The training parameters of the experimental model are configured as follows: the number of epochs is 300, the input image size is 640×640 , the batch size is 32, and the optimizer is set to SGD.

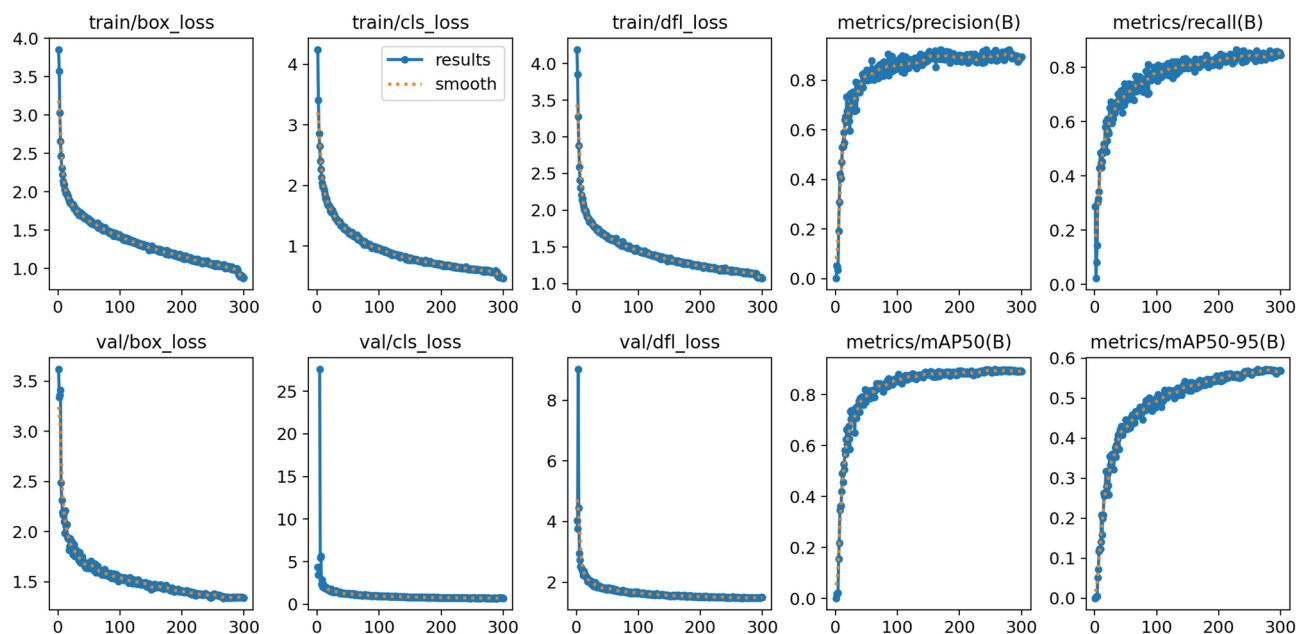
Model evaluation metrics

The evaluation metrics used in this paper include Precision, Recall, mean average precision (mAP50) at an IoU threshold of 0.5, model size (Params), and the number of floating-point operations (GFLOPs). Precision measures the accuracy of the detection results, while Recall measures the model's ability to detect all relevant instances. mAP50 represents the average precision of all categories. Model size reflects the complexity and scale of the model, and GFLOPs indicate the computational cost. The calculation formulas are as follows:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (4)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (5)$$

Model	mAP50/%	Precision/%	Recall/%	Params/MB	FLOPs/G
YOLOv8n	88.9	89.8	83.8	6.0	8.1
YOLOv8n+DynamicConv	89.4	86.7	85.4	5.2	6.7
YOLOv8n+DynamicConv+SEAM	89.0	90.3	82.9	5.5	7.0
DSS-YOLO	89.5	90.2	85.4	5.8	7.1

Table 1. DSS-YOLO ablation experiment.**Fig. 4.** DSS-YOLO training results figure.

$$\text{mAP} = \frac{1}{n} \sum_{k=1}^n \text{AR}_k \quad (6)$$

where TP (true positives) denotes samples that are positive and are predicted to be positive, FP (false positives) denotes samples that are negative but are predicted to be positive, FN (false negatives) denotes samples that are positive but are predicted to be negative, n represents the number of classes, and AP_k represents the average accuracy of k classes.

Ablation experiments and analysis

In order to verify the effectiveness of each improved module in the algorithm of this paper, 4 groups of ablation experiments were conducted on the mytest-hrswj data set. The results of the ablation experiments are shown in Table 1. The first group used the original YOLOv8n model for benchmark experiments. The second group replaced the original C2f module in the backbone network with the DynamicConv structure. Although the model accuracy was reduced by 3.1%, the recall rate and mAP50 of the model were increased by 1.6% and 0.5% respectively, and the model size and calculation amount were reduced by 8.3% and 17.0% respectively, which shows that the DynamicConv structure can improve some performance while reducing the amount of calculation. The third group of experiments is based on the second group, and the SEAM attention mechanism is introduced in the head network. Its accuracy is improved by 3.6%, but the recall rate and mAP50 are reduced, and the model size and calculation amount are slightly increased. The fourth group of experiments introduces SPPELAN to replace the original SPPF based on the third group, which improves the recall rate and mAP50 by 2.5% and 0.5% respectively, the accuracy is reduced by a small 0.1%, and the model size and calculation amount are also slightly increased. In general, compared with YOLOv8n, the model size and computational complexity are reduced by 3.3% and 12.3% respectively, the precision is increased by 0.4%, the recall rate is increased by 1.6%, and the mAP50 is increased by 0.6%.

The training results of DSS-YOLO are shown in Fig. 4. From the decrease in loss, it is evident that the model exhibits good convergence during training, and the validation set loss also decreases, indicating no signs of overfitting. The gradual improvement in precision, recall, and mAP demonstrates the continuous optimization of the model's detection performance, with precision ultimately reaching 0.90, recall approaching 0.83, and mAP50 and mAP50-95 nearing 0.90 and 0.55, respectively.

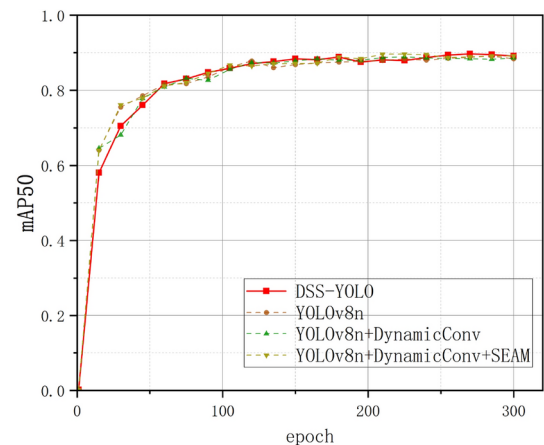


Fig. 5. mAP50 curve changes with the number of training rounds.

Model	Precision/%	mAP50/%	Recall/%	Params/MB	FLOPs/G
YOLOv6s	89.5	88.3	84.6	31.4	44.0
YOLOv8s	90.0	90.8	87.5	21.5	28.4
YOLOv8n	89.8	88.9	83.8	6.0	8.1
YOLOv5n	89.3	88.6	83.0	5.1	7.1
DSS-YOLO	90.2	89.5	85.4	5.8	7.1

Table 2. Comparison of different algorithms.

To visually demonstrate the average precision of each model, this paper charted the mAP50 progression across training rounds, as depicted in Fig. 5. The graph reveals that the YOLOv8n + DynamicConv + SEAM + SPPELAN model achieves the highest mAP50, highlighting its superiority in enhancing model performance. The YOLOv8n + DynamicConv + SEAM configuration ranks second, while YOLOv8n + DynamicConv shows a slight improvement over the basic YOLOv8n.

Comparative experiments and analysis

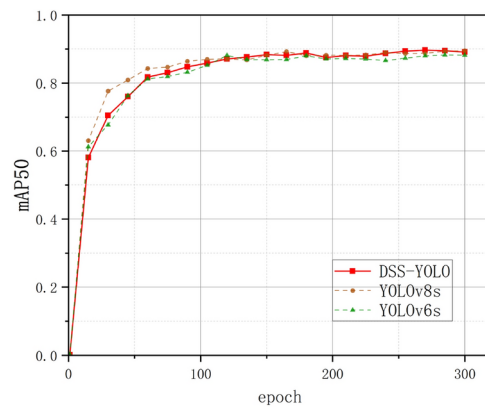
To further verify the performance of the DSS-YOLO model described in this paper, comparisons were made with other mainstream target detection algorithms on the mytest-hrsdw dataset. Precision, Recall, mAP50, Params, and GFLOPs were used as evaluation metrics. The specific models and comparison results are shown in Table 2.

As shown in Table 2, compared to other models, DSS-YOLO exhibits the best overall precision, ranking second in mAP50. When compared to the larger models, YOLOv6s and YOLOv8s, DSS-YOLO achieves the highest precision, with its mAP50 only 1.3% lower than YOLOv8s, which has over three times the number of parameters. When compared to YOLOv5n and YOLOv8n, which have similar parameter counts, the DSS-YOLO model outperforms both in all metrics. Obviously, DSS-YOLO better balances the requirements of model computational complexity and high precision and has high real-time detection capabilities. The results show that DSS-YOLO is the best way to detect fire targets among the five models, effectively reducing the amount of computation and maintaining good recognition precision, which can meet the requirements of real-time detection of fires.

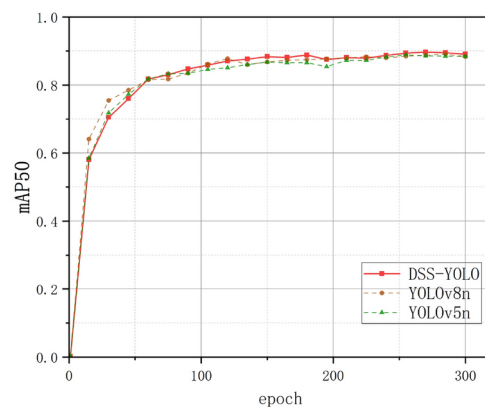
In summary, DSS-YOLO, through model optimization, effectively balances the trade-off between accuracy and detection speed, making it well-suited to meet the efficient and real-time processing requirements for fire detection. Moreover, it demonstrates strong robustness in recognizing fire images at various scales. This study holds significant practical value for efficient detection and recognition in multi-scale fire scenarios.

Figure 6 compares the mAP50 curves of DSS-YOLO with those of five other models over training epochs. Figure 6a shows the mAP50 curves during the training cycles of DSS-YOLOv8 and two models with larger parameter sizes, YOLOv8s and YOLOv6s. For YOLOv6s, DSS-YOLO maintains a consistent advantage throughout the training. For YOLOv8s, although DSS-YOLO's convergence speed is slower than YOLOv8s in the first 100 epochs, the performance gap between the two models reaches its minimum when the training cycles reach 150 epochs, with DSS-YOLO even surpassing YOLOv8s slightly after 200 epochs. On the other hand, Fig. 6b illustrates the mAP50 curves for DSS-YOLO compared to two models with similar parameter sizes, YOLOv8n and YOLOv5n. After 90 epochs, DSS-YOLO consistently surpassed both models, further highlighting the stable performance improvements of DSS-YOLO throughout the training period.

Figure 7 illustrates the comparative detection performance of the original YOLOv8n model versus the DSS-YOLO model proposed in this paper on the mytest-hrsdw dataset. The figure reveals that the detection capability of the original model is somewhat restricted, failing to identify certain subtle and obscured features. Despite its



(a) mAP50 curves during training for DSS-YOLO, YOLOv8s, and YOLOv6s



(b) mAP50 curves during training for DSS-YOLO, YOLOv8n, and YOLOv5n

Fig. 6. fig:mAP50 Variation curve with training epochs (a) mAP50 curves during training for DSS-YOLO, YOLOv8s, and YOLOv6s. (b) mAP50 curves during training for DSS-YOLO, YOLOv8n, and YOLOv5n.

constrained size, the DSS-YOLO model is able to detect these subtle and obscured fire-related features, generally outperforming the original model.

This paper presents example images of fire detection using the DSS-YOLO model, showcasing its ability to detect fire across various environments. The experimental results are shown in Fig. 8, which includes obstructed objects, small objects, and some conventional scenarios.

In these examples, the model demonstrates precise detection of small and obstructed fire, as seen in Fig. 8a and b. It also effectively identifies both obstructed objects and smaller fire, as illustrated in Fig. 8c and d. For standard fire detection tasks, such as those in Fig. 8e, f, g, and h, the model generates accurate and comprehensive bounding boxes, precisely delineating the targets without noticeable positional errors or omissions. Overall, the model excels in detecting obstructed and small targets, while also performing robustly in general fire detection scenarios.

Discussion

This paper is based on improvements to YOLOv8n, resulting in the successful development of the DSS-YOLO model. It significantly improves detection accuracy for occluded objects and small targets while reducing computational complexity. This improvement is particularly important in practical applications, especially for early fire detection, as timely identification and response to small flames can greatly reduce the potential harm caused by fires.

In this paper, the C2f module in the model was replaced with the DynamicConv module, effectively reducing the computational load, which means the model can run on hardware with limited computational resources, lowering deployment costs. Additionally, the application of the SEAM attention mechanism helps the model

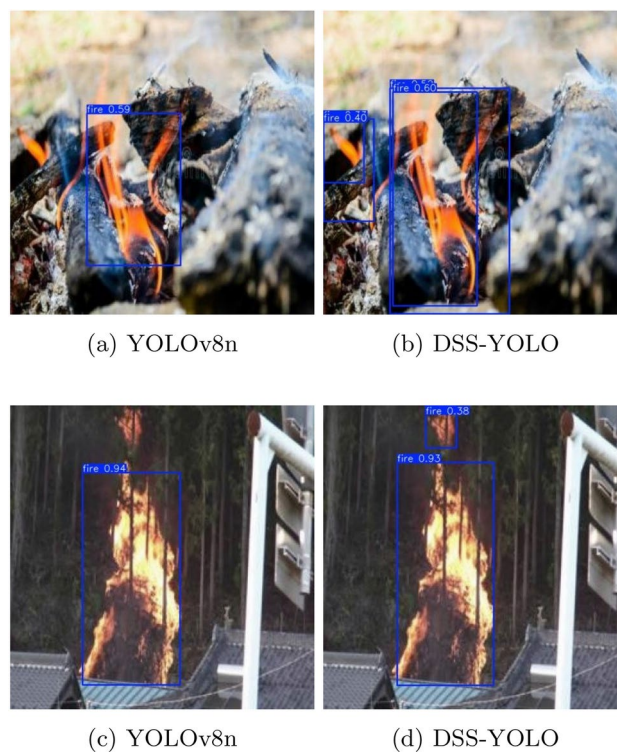


Fig. 7. Comparison of YOLOv8n and DSS-YOLO on fire detection. (a) YOLOv8n result 1. (b) DSS-YOLO result 1. (c) YOLOv8n result 2. (d) DSS-YOLO result 2.

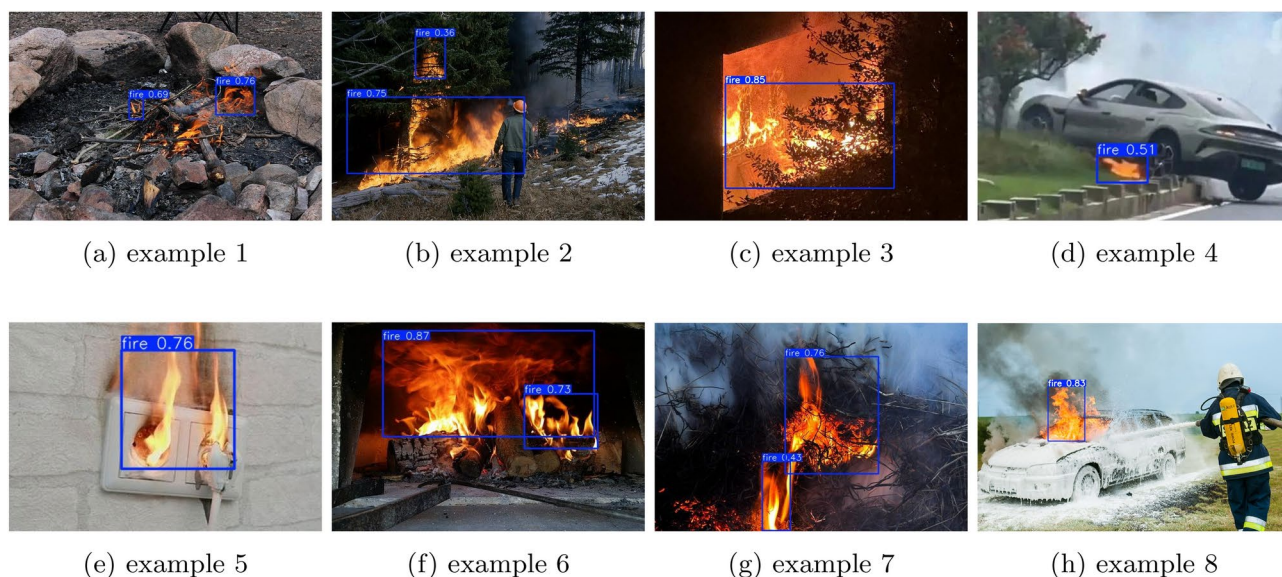


Fig. 8. Examples of fire detection in various environments by DSS-YOLO.

focus better on key areas, which is crucial for detecting occluded targets. The introduction of the SPPELAN module further enhances the model's ability to detect multi-scale targets.

Previous research has demonstrated the superior performance of the YOLO series models in real-time object detection, but they still have limitations when dealing with small and occluded targets. This paper addresses these limitations with targeted improvements. For instance, previous work may not have effectively handled the detection of occluded fire objects or adequately balanced computational load and accuracy. The design of DSS-YOLO takes these factors into full consideration, making the model perform exceptionally well in object detection tasks under complex environments.

Given the importance of fire detection, especially in densely populated areas such as schools, hospitals, or museums housing valuable artifacts, quickly and accurately identifying signs of fire is crucial to reducing the loss of life and property. The DSS-YOLO model not only provides a technological upgrade to existing fire monitoring systems but also offers new perspectives for the development of future intelligent safety monitoring systems.

Although the DSS-YOLO model has shown excellent performance in the current study, there are still many directions worth exploring. Firstly, the model's generalization ability requires further validation, particularly for different types of fire scenarios, including but not limited to industrial fires and forest fires. Secondly, further compressing the model size while maintaining high accuracy, enabling it to run on a broader range of devices, remains a critical research challenge. Lastly, integrating other sensor data (such as temperature, smoke concentration, etc.) with visual information could further enhance the overall performance of fire detection systems.

Conclusion

This paper proposes a DSS-YOLO model for fire object detection tasks, which improves detection accuracy while maintaining low computational load and model size, demonstrating practical application value. Firstly, all C2f modules in the Backbone section were replaced with DynamicConv modules to reduce computational complexity while retaining good feature extraction capabilities. Then, the SEAM module was added after each C2f layer in the Head section to enhance detection of occluded objects and small targets. Finally, the SPPELAN module was added to the end of the Backbone section to improve the model's ability to detect targets of different scales. Extensive experiments on the mytest-hrswj dataset validated the effectiveness of the improved algorithm. Compared with the original YOLOv8n, the proposed DSS-YOLO model achieved an increase of 0.6% in mAP and 1.6% in Recall, while reducing model size and FLOPs by 3.4% and 12.3%, respectively, meeting the requirements for real-time fire detection. In comparison with YOLOv8n, DSS-YOLO shows better performance in detecting occluded and less obvious targets. Future research will focus on further improving the model's generalization ability and enhancing overall detection performance in different environments.

This study has achieved significant results in the detection of small and occluded objects. However, DSS-YOLO still faces challenges in recognizing flames under extreme conditions, such as overexposure or low resolution. Additionally, its performance in flame detection is suboptimal in scenes with backgrounds similar to flames, such as sunsets or red-colored environments. In future work, model robustness can be improved through techniques like quantization and pruning, combined with multimodal fusion, such as temperature, humidity, or smoke sensor data, to enhance adaptability in complex environments and provide more reliable technical support for intelligent fire safety systems.

Data availability

The dataset used in this paper, mytest-hrswj, is publicly available on the Roboflow platform at <https://universe.roboflow.com/mytyolov8/mytest-hrswj/dataset>. Accessed on: 2024-08-17.

Appendix A: Pseudocode

```

1: Step 1: Data Preparation
2: image_dir = './dataset/fire/'
3: data = Load(image_dir)
4: Step 2: Dataset Splitting
5: train_data, val_data, test_data = Split-
   Dataset(data, train_ratio=0.8, val_ratio=0.1,
   test_ratio=0.1)
6: Step 3: Data Augmentation
7: augmented_train_data = Augment-
   Data(train_data, methods=['HSV', 'rotate',
   'translate', 'scale', 'flip', 'mosaic'])
8: Step 4: Model Initialization
9: model = LoadModel('DSS-YOLO')
10: Step 5: Set Hyperparameters
11: hyperparameters = { 'epochs': 100, 'batch':
   32, 'imgsz': 640, 'optimizer': 'SGD', 'patience':
   20 }
12: Step 6: Model Architecture
13: model_layer(Conv(64, 3, 2))
14: model_layer(Conv(128, 3, 2))
15: model_layer(DynamicConv(128, 3, 2))
16: ...
17: model_layer(SPPELAN(1024, 256))
18: ...
19: model_layer(Upsample(scale_factor=2,
   mode='nearest'))
20: model_layer(Concat(1))
21: model_layer(C2f(512))
22: model_layer(SEAM())
23: ...
24: model_layer(Detect(num_classes))
25: Step 7: Train the Model
26: Train(model, data=augmented_train_data,
   hyperparameters=hyperparameters)
27: Step 8: Save Trained Model
28: SaveModel(model, 'DSS-YOLO.pt')
29: Step 9: Image Visualization During
   Training
30: VisualizeImages(augmented_train_data,
   phase='train')
31: Step 10: Test the Model
32: test_results = Test(model, data=test_data)
33: VisualizeImages(test_data, phase='test')
34: Step 11: Validate the Model
35: val_results = Validate(model, data=val_data)
36: VisualizeImages(val_data, phase='val')

```

Algorithm 1. DSS-YOLO pseudo code

Received: 7 January 2025; Accepted: 5 March 2025

Published online: 15 March 2025

References

1. Cvetković, V. M. et al. Fire safety behavior model for residential buildings: Implications for disaster risk reduction. *Int. J. Disaster Risk Reduct.* **76**, 102981. <https://doi.org/10.1016/j.ijdrr.2022.102981> (2022).
2. Fire, C.N., & Administration, R. The average number of fires in China exceeded 3,000 per day in the first half of 2023. <https://www.119.gov.cn/qmxfkg/sjtj/2023/38420.shtml>. (2024)
3. Sulthana, S. F. et al. Review study on recent developments in fire sensing methods. *IEEE Access* **11**, 90269–90282. <https://doi.org/10.1109/ACCESS.2023.3306812> (2023).
4. Cai, G., Zheng, X., Gao, W. & Guo, J. Self-extinction characteristics of fire extinguishing induced by nitrogen injection rescue in an enclosed urban utility tunnel. *Case Stud. Therm. Eng.* **59**, 104478. <https://doi.org/10.1016/j.csite.2024.104478> (2024).
5. Shri Vidhatri, M. M., Sivaramakrishna, M., & Chitrakumar, S. Design of an innovative smoke sensor and fire alarm panel. In *Proceedings of the 2022 Third International Conference on Intelligent Computing Instrumentation and Control Technologies (ICICT)*, 1365–1369 (IEEE, 2022). <https://doi.org/10.1109/ICICT54557.2022.9917916>.
6. Hendel, I.-G. & Ross, G. M. Efficacy of remote sensing in early forest fire detection: A thermal sensor comparison. *Can. J. Remote Sens.* **46**(4), 414–428. <https://doi.org/10.1080/07038992.2020.1776597> (2020).
7. Solórzano, A. et al. Early fire detection based on gas sensor arrays: Multivariate calibration and validation. *Sens. Actuators B Chem.* **352**, 130961. <https://doi.org/10.1016/j.snb.2021.130961> (2022).
8. Agarwal, N. & Rohilla, Y. Flame sensor based autonomous firefighting robot. In *Proceeding of Fifth International Conference on Microelectronics, Computing and Communication Systems* (eds Nath, V. & Mandal, J. K.) 641–655 (Springer, 2021). https://doi.org/10.1007/978-981-16-0275-7_52.
9. Cai, G., Zheng, X., Guo, J. & Gao, W. Real-time identification of borehole rescue environment situation in underground disaster areas based on multi-source heterogeneous data fusion. *Saf. Sci.* **181**, 106690. <https://doi.org/10.1016/j.ssci.2024.106690> (2025).
10. Khan, F. et al. Recent advances in sensors for fire detection. *Sensors* **22**(9), 3310. <https://doi.org/10.3390/s22093310> (2022).
11. Sharifani, K., & Amini, M. Machine Learning and Deep Learning: A Review of Methods and Applications. <https://papers.ssrn.com/abstract=4458723>. Accessed 2 Oct 2024 (2023)
12. Zhang, Y., Xiao, X., Wang, W., Wang, C., Jin, X., & Wang, Y. Improved lightweight flame smoke detection algorithm for yolov8n. In *2024 39th Youth Academic Annual Conference of Chinese Association of Automation (YAC)*, 1537–1542 (IEEE, 2024). <https://doi.org/10.1109/YAC63405.2024.10598605>.
13. Wang, Z. Enhanced fire detection algorithm for chemical plants using modified yolov9 architecture. In *2024 9th International Conference on Electronic Technology and Information Science (ICETIS)*, 22–25 (IEEE, 2024). <https://doi.org/10.1109/ICETIS61828.2024.10593793>.
14. Zhao, L., Zhi, L., Zhao, C. & Zheng, W. Fire-yolo: A small target object detection method for fire inspection. *Sustainability* **14**(9), 4930. <https://doi.org/10.3390/su14094930> (2022).
15. Xue, Z., Lin, H. & Wang, F. A small target forest fire detection model based on yolov5 improvement. *Forests* **13**(8), 1332. <https://doi.org/10.3390/f13081332> (2022).
16. Shao, D. et al. Yolov7scb: A small-target object detection method for fire smoke inspection. *Fire* **8**(2), 62. <https://doi.org/10.3390/fire8020062> (2025).
17. Jocher, G., Qiu, J., & Chaurasia, A. Ultralytics YOLO. Python. <https://github.com/ultralytics/ultralytics>. Accessed 3 Oct 2024 (2023)
18. Sohan, M., Sai Ram, T. & Rami Reddy, C. V. A review on yolov8 and its advancements. In *Data Intelligence and Cognitive Informatics* (eds Jacob, I. J. et al.) 529–545 (Springer, 2024). https://doi.org/10.1007/978-981-99-7962-2_39.
19. Terven, J., Córdova-Esparza, D.-M. & Romero-González, J.-A. A comprehensive review of yolo architectures in computer vision: From yolov1 to yolov8 and yolo-nas. *Mach. Learn. Knowl. Extr.* **5**(4), 1680–1716. <https://doi.org/10.3390/make5040083> (2023).
20. Jiang, W., Yang, L. & Bu, Y. Research on the identification and classification of marine debris based on improved yolov8. *J. Mar. Sci. Eng.* **12**(10), 1748. <https://doi.org/10.3390/jmse12101748> (2024).
21. Han, K., Wang, Y., Guo, J., & Wu, E. ParameterNet: Parameters Are All You Need (2024). [arXiv:2306.14525](https://arxiv.org/abs/2306.14525)
22. Yu, Z., Huang, H., Chen, W., Su, Y., Liu, Y., & Wang, X. YOLO-FaceV2: A Scale and Occlusion Aware Face Detector (2022). [arXiv:2208.02019](https://arxiv.org/abs/2208.02019)
23. Wang, C.-Y., Yeh, I.-H., & Liao, H.-Y.M. YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information (2024). [arXiv:2402.13616](https://arxiv.org/abs/2402.13616)
24. MytYolov8: Mytest Dataset. Roboflow. visited on 2024-10-09 (2024). <https://universe.roboflow.com/mytYolov8/mytest-hrswj>

Acknowledgements

The authors are grateful to the support of the NSFC (62372494), Guangdong Virtual Teaching Group (2022002), Engineering Centre Project (2024GCZX001), Special Talent Training Program (2024001) and Key Disciplines Project (2024ZDJS142).

Author contributions

Conceptualization, X.Fu and H.Wang; methodology, X.Fu and H.Wang; software, H.Wang; validation, H.Wang, X.Fu, and Z.Yu; writing—original draft preparation, H.Wang; writing—review and editing, H.Wang, X.Fu, Z.Yu; visualization, H.Wang, Z.Yu; supervision, X.Fu, Z. Zeng; project administration, X.Fu; funding acquisition, X.Fu. All authors have read and agreed to the published version of the manuscript.

Funding

This research was funded by the National Natural Science Foundation of China (No. 62372494), Key Disciplines Project (No. 2022ZDJS139).

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to X.F.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025